



UpgradeShield: Detecting logic-state in-consistencies in smart contract upgrades

Wei Li¹ · Yuxin Su¹ · Yuhong Nan¹ · Kaiwen Ning^{1,2} · Jingwen Zhang^{1,2} · Jiajing Wu¹ · Zibin Zheng¹

Received: 15 January 2025 / Accepted: 14 February 2026

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

Smart contracts are integral to decentralized finance and other blockchain applications, leveraging blockchain's immutability to ensure security and transparency. However, logic-state inconsistencies (referred to as LSI in this paper) can emerge during contract upgrades, potentially causing transaction failures, economic imbalances, or security vulnerabilities. A notable example of LSI's impact is the 190 million Nomad bridge exploit, which underscores the significant risks associated with inconsistent upgrades. Existing tools such as static analysis, fuzz testing, and formal verification struggle to address these inconsistencies due to the complexity and dynamic nature of LSI. In this work, we present UpgradeShield, a framework that detects LSI in smart contract upgrades by integrating transaction replay, code clone analysis, and differential testing. Historical transactions are replayed on both old and new contract versions to capture execution results and storage states, while code clone analysis identifies whether logic changes require state consistency. Combining these insights, differential testing compares the behaviors of both versions under identical conditions to detect inconsistencies introduced during the upgrade process. We evaluated UpgradeShield on 2,015 proxy-based contract upgrades, demonstrating its effectiveness in detecting upgrade-related bugs. This work provides a practical solution for ensuring the security and reliability of smart contract upgrades, and we release the source code and dataset to support further research.

Keywords Ethereum · Smart contract upgrade · Differential test · Record-replay testing

1 Introduction

Smart contracts, extensively used in decentralized finance and other blockchain-based applications, rely on blockchain's immutability and decentralization to ensure transparency and security. However, as smart contracts gain broader adoption, they increasingly encounter significant security threats, with attackers potentially exploiting contract vulnerabilities to launch attacks that result in financial losses. Consequently, contract upgrades have become indispensable for addressing these threats, improving security, refining token models, or introducing new functionalities. To facilitate contract upgrades, multiple Ethereum Improvement Proposals (EIPs) have introduced various strategies (Palladino et al. 2019; Gabriel Barros 2019; Mudge 2020), such as contract migration, the eternal storage model, and proxy patterns. Meanwhile, frameworks like OpenZeppelin (Openzeppelin 2023) provide ready-made contract templates to expedite the upgrade process. Nonetheless, ensuring consistency between the new contract's logic and the old contract's state remains a major challenge. Any discrepancy or design flaw in the upgraded contract can result in significant economic losses.

In this paper, we define Logic-state inconsistency (called LSI in this paper) as a semantic divergence in storage state or observable behavior that occurs when the same input is applied to both pre-upgrade and post-upgrade versions of a smart contract. These inconsistencies arise when the upgraded logic fails to preserve behavioral equivalence under the assumptions of the existing historical state. Even if each version behaves correctly in isolation, inconsistencies may emerge due to subtle differences in logic, calculation precision, or control flow. LSIs are particularly dangerous because they often manifest silently—without triggering standard security checks—yet may lead to transaction failures, economic imbalances, or exploitable behaviors (Halborn 2022; Finance 2021). The gravity of this issue has been thoroughly demonstrated by several historical incidents. For example, in July 2022, hackers exploited a storage collision vulnerability between proxy contracts and logic contracts to successfully upgrade a contract and steal 18 million AUDIO tokens (Cointelegraph 2022). In August of the same year, a logic flaw introduced during the upgrade of the Nomad bridge protocol led to a financial loss of \$190 million. These examples underscore the risks of overlooking consistency between logic and state during contract upgrades—whether through neglecting historical states or introducing unreasonable rules—ultimately causing significant financial damages.

Existing vulnerability detection tools struggle to capture LSIs for several reasons. Traditional static analysis tools, such as Slither and Mythril (Feist et al. 2019; Mueller 2018), primarily rely on rule matching and static inspection. While they are capable of identifying vulnerabilities like reentrancy attacks and access control flaws, they struggle to handle complex logic inconsistencies involving dynamic state interactions. Contract upgrades introduce more complicated scenarios, where each version of the contract may function correctly within its own context, but due to differences in logic and storage between versions, LSI may still arise. Although certain obvious storage layout conflicts can be detected by static analysis tools (e.g., Slither Feist et al. 2019; Bodell et al. 2023; Ruaro et al. 2024a), existing tools find it difficult to capture bugs once contract logic is involved. Furthermore, traditional unit testing

methods depend on manual writing, making it challenging to cover all possible interaction paths. Although fuzzing can automatically generate test inputs, its inherently random nature fails to effectively detect issues when contract upgrades involve complex states and deep call chains (Xue et al. 2022; Torres et al. 2021; Jiang et al. 2018). Formal verification (Kalra et al. 2018; Wang et al. 2019), while capable of providing more precise validation, faces significant challenges in practical application due to its high cost and inadequate adaptability to dynamic upgrade scenarios. Therefore, existing tools exhibit notable shortcomings in the process of smart contract upgrades, particularly in handling LSI, necessitating new solutions to address this challenge.

Addressing LSI involves multiple technical challenges. First, **complexity of simulating logic and state** is a primary issue. After a contract upgrade, the new logic must operate on the old state, which is typically generated by intricate historical transactions and interactions. When the call chain is deep, reproducing the process of logic and state interactions becomes exceedingly difficult, and traditional fuzz testing struggles to effectively simulate complex call sequences, resulting in insufficient test coverage and an inability to comprehensively assess the contract's performance under diverse states. Second, **complexity of understanding logic changes** poses another significant challenge. Contract upgrades involve adjustments to business rules, additions of new functionalities, or optimizations of existing features. These changes often implicitly depend on the characteristics of the old state. Accurately identifying which changes need to maintain compatibility with the old state and which do not is a complex task in contract upgrades, especially when there are significant differences between the new and old logic. Lastly, **determining the authenticity of behavioral differences** is also a major difficulty. When executing the same transaction on new and old versions of a contract, behavioral differences may arise from state dependencies or logic optimizations. Accurately distinguishing whether these differences are due to LSI, while avoiding false positives or negatives, is crucial for ensuring the safety and stability of contract upgrades.

To address LSI in smart contract upgrades, we propose a comprehensive approach that integrates transaction replay, code clone analysis, and differential testing. This method begins with utilizing transaction replay techniques to execute historical transactions from the legacy contract on both the old and new contract versions, thereby obtaining fine-grained execution results and storage states. Subsequently, code clone analysis is employed to compare the codebases of the new and old contract versions using binary code clone detection techniques, determining whether the new contract logic needs to maintain consistency with the old state. Finally, by combining the results from transaction replay and code clone analysis, differential testing is conducted to compare the execution behaviors of the new and old contract versions under the same historical states, thereby detecting potential LSI.

In this paper, we propose a novel framework named UpgradeShield to detect LSI. Our framework consists of four modules: Historical Information Collection (HIC), Result Tracking Generation (RTG), Refactoring Consistency Check (RCC), and Result Tracking Comparison (RTC).

In HIC, historical transactions are replayed, and the input and output states of each transaction are recorded, providing authentic historical data for subsequent analysis. The RTG module incrementally replays different versions of the contract, recording

the execution trace information for each version. The RCC module compares functional code clones in different contract versions to determine whether the new and old versions maintain state consistency. In this work, if two versions of a smart contract require consistent states, we consider them to exhibit “refactoring consistency”. Finally, in RTC, the execution results of different contract versions are compared to determine whether LSI exists during the contract upgrade process.

To evaluate the effectiveness of UpgradeShield, we collect a comprehensive dataset of 2,015 smart contract upgrade cases using proxy mode by accessing Etherscan and replaying historical transactions. We obtained bytecode for both the original and upgraded contract during contract creation and execution by replaying historical transactions, and evaluated the effectiveness of UpgradeShield using these transactions. We manually verified 178 of the proxy contract upgrade cases, and the result shows that UpgradeShield is reliable in the context of smart contract upgrades (see Section 5.3 for details). In addition, to achieve better experimental results, we tested different parameter thresholds for refactoring consistency in the experimental section and selected the best-performing parameters as the parameters for UpgradeShield (see Section 5.4 for details).

In the spirit of open source, we release the source code of UpgradeShield, as well as the corresponding dataset used in our research. A preliminary version of UpgradeShield and our dataset (i.e., the proxy addresses and their corresponding logic addresses) can be accessed via the following repository: <https://github.com/xueying6371/UpgradeShield>. The contributions of this work are summarized as follows:

- To the best of our knowledge, this is the first vulnerability detection work focused on logic-state inconsistencies in smart contract upgrades.
- We propose and develop UpgradeShield, a new framework for improving the confidence that the process of contract upgrade is sound.
- We conduct extensive experiments to evaluate the effectiveness of UpgradeShield. Results show that using UpgradeShield to check the process of contract upgrade is necessary.

The structure of the remaining sections is organized as follows: Section 2 provides background information and presents a motivating example to elucidate the fundamental concept of UpgradeShield. Section 3 identifies the principal challenges and corresponding solutions, offering an overview of UpgradeShield. Section 4 elaborates on the design of UpgradeShield, while Section 5 details its implementation. Subsequently, Section 6 outlines the experimental setup and evaluation of UpgradeShield, accompanied by a discussion of its limitations in Section 7. Section 8 surveys related work, and Section 9 concludes the paper.

2 Background and motivation

2.1 Background

In this section, we provide background of the Ethereum blockchain, world State, EVM, smart contracts, smart contract upgrade strategies, and differential testing.

Ethereum and the Ethereum Virtual Machine (EVM) Blockchain is a decentralized and tamper-proof data ledger that guarantees security, implemented as a chain of blocks. Ethereum is a public decentralized platform that provides the ability to deploy smart contracts and is widely recognized as marking the advent of “blockchain 2.0”. According to the Ethereum Yellow Paper (Wood 2020), blocks are mined by miners and accepted by the consensus protocol (i.e., POS). Each block includes a header with the parent block reference and verification metadata, and a body containing an ordered list of transactions submitted by network participants. Ethereum supports three main types of transactions. A transfer transaction moves assets between accounts. A creation transaction deploys a new contract by submitting its bytecode and initializing its state. An invocation transaction executes a contract function using supplied parameters and access to global state. The world state records a mapping from Ethereum addresses to account states, and it evolves as transactions are executed. It can be viewed as the persistent storage of a decentralized computer. All account information is maintained within the world-state structure, which provides the foundation for defining the historical states used in our transaction-replay analysis.

The Ethereum network uses a gas mechanism to ensure that contract execution terminates and to incentivize transaction processing. Each EVM instruction has an associated gas cost that reflects its resource usage. If the provided gas is insufficient, the current call fails.

The execution of all contracts is handled by the Ethereum Virtual Machine (EVM), a stack-based, Turing-complete interpreter that executes contract bytecode deterministically. Each byte in the bytecode represents either an operation code or an operand encoded in hexadecimal form. To maintain consensus, the EVM enforces deterministic execution such that identical pre-states and inputs produce identical post-states and outputs on all honest nodes. Non-deterministic operations, such as floating-point arithmetic, are prohibited to prevent divergence across executions. Deterministic execution provides the foundation for reproducible evaluation in UpgradeShield, enabling us to interpret observed behavioral differences during replay and differential testing as potential LSIs rather than effects of nondeterministic execution.

Smart contract A smart contract is a program deployed on the blockchain that automatically executes predefined logic when invoked by transactions. Solidity is the dominant high-level language used to develop smart contracts on Ethereum. It is Turing-complete and supports control structures such as loops, conditionals, and function calls, as well as object-oriented constructs and complex data types.

When compiled, a Solidity contract produces two main bytecode artifacts: the deployment bytecode (`createbin`) and the runtime bytecode (`runtimebin`). The deployment bytecode runs only during contract creation, executing the constructor to initialize storage variables (such as the contract owner) and returning the runtime program that will persist on-chain. The runtime bytecode is stored in the contract’s account and is executed on subsequent invocations. The compiler also appends auxiliary metadata, including source hashes and ABI references, which support off-

chain verification but are not executed by the EVM. After deployment, the code of a contract resides in the account's `Code` field, while persistent variables are maintained in `Storage`. This execution-and-storage model makes contract behavior reproducible and supports systematic differential analysis in UpgradeShield.

Contract upgrade strategies Smart contracts on Ethereum are generally considered immutable because, once deployed, they cannot be modified. To address this limitation, the Ethereum community has proposed several approaches for deploying upgraded contracts (Vitalik Buterin 2017; Jorge Izquierdo 2018; Gabriel Barros 2019; Palladino et al. 2019; Mudge 2018, 2020), including contract migration and separating contract logic from data storage (Openzeppelin 2023; Apostolou 2023).

Contract migration involves discarding the old contract and redeploying a new one while transferring the existing state. The original contract can be destroyed via the `self-destruct` function, and a replacement may be deployed to the same address using `create2`. However, this process demands significant effort: developers must extract and record all internal states from the old contract, implement a transfer function in the new contract, and execute several transactions to restore the contract's state—all of which incur considerable labor and gas costs.

Alternatively, separating logic from data can be achieved through two main approaches: the eternal storage model and the proxy model. The distinction lies in whether users interact with a dedicated logic contract or a storage-focused contract. In the eternal storage model, users communicate with the logic contract that handles business operations, which uses standard `CALL` operations to access a separate, immutable storage contract (Openzeppelin 2026). This explicit separation allows logic upgrades while preserving storage, but incurs higher gas costs due to cross-contract calls (Wood 2014). In contrast, the proxy model allows users to interface directly with a data-storing “proxy” contract, which employs the `DELEGATECALL` (Jorge Izquierdo 2018) instruction to invoke the business logic from another contract. Here, the proxy contract remains immutable and serves as the primary entry point for user transactions, whereas the logic contract houses the core functionality and does not directly handle user interactions. Upgrading simply involves updating the proxy contract to reference a new logic contract. Extensions of the proxy model include the diamond pattern (Mudge 2020), which employs multiple proxy contracts, and the strategy pattern, which embeds certain logic in the proxy contract while delegating other functionalities to additional logic contracts. The proxy mode simplifies state migration complexity and provides a more flexible contract upgrade mechanism.

Differential testing Differential testing is a technique that detects potential bugs by comparing the outputs or behaviors of multiple implementations or versions under the same inputs. In the context of software reliability, tools like Csmith (Yang et al. 2011) generate randomized test cases and compile them using different compilers—differences in output signal potential implementation flaws. Similarly, CompDiff (Li and Su 2023) runs the same input across multiple compiler binaries (e.g., GCC vs. Clang at various optimization levels) to detect unstable or undefined behaviors that manifest as divergent outputs.

By treating each compiler—or, analogously, each pre-upgrade vs. post-upgrade smart contract version—as a separate “implementation,” differential testing effectively identifies subtle semantic deviations. It excels especially in scenarios where traditional tests fail to explore deep execution paths, offering a robust, automation-friendly mechanism to detect behavioral inconsistencies introduced by code changes or software upgrades.

2.2 Key definitions

Contract semantics Let C denote a smart contract with state space S and input space I . We model the semantics of C as a transition function

$$\llbracket C \rrbracket : S \times I \rightarrow S' \times O,$$

where O denotes the externally observable outcome (e.g., return value and success/revert status). For any $s \in S$ and $i \in I$, $\llbracket C \rrbracket(s, i) = (s', o)$ yields the next state s' and outcome o .

Upgrade semantics Consider an upgrade from contract C_{old} to C_{new} , whose semantics are $\llbracket C_{\text{old}} \rrbracket$ and $\llbracket C_{\text{new}} \rrbracket$, respectively. Upgrades may extend functionality or optimize performance for new inputs. When an upgrade is intended as refactoring, the developer’s goal is to preserve behavior on pre-upgrade states under identical inputs, so that existing on-chain state remains compatible with the new code. This intent is operationalized by the refactoring consistency check below.

Refactoring Consistency (RC) RC serves as a structural precondition that approximates refactoring intent. Let $\mathcal{R}$ be the set of code regions in C_{old} that our matcher aligns to corresponding regions in C_{new} (e.g., matched functions). Let $\text{sim}(\cdot, \cdot)$ denote a normalized similarity measure over aligned regions. RC holds if the aligned regions remain highly similar, i.e., $\text{sim}(\mathcal{R}(C_{\text{old}}), \mathcal{R}(C_{\text{new}})) \geq \theta$ for a configurable threshold θ , while allowing additive extensions outside $\mathcal{R}$. Under RC, the upgrade is expected to preserve behavior on historical states.

Logic-State Inconsistency (LSI) Given an upgrade pair that satisfies RC, an LSI exists if there is a historical state-input pair (s, i) , reconstructed by replaying pre-upgrade transactions, such that

$$\llbracket C_{\text{old}} \rrbracket(s, i) \neq \llbracket C_{\text{new}} \rrbracket(s, i).$$

This condition means the two versions yield different outputs or induce different storage updates under identical historical conditions. Such inconsistencies constitute an LSI and indicate that the new logic conflicts with assumptions encoded in the pre-upgrade state, signaling potential violations of intended correctness with possible downstream effects on fairness, safety, or economic outcomes.

2.3 Motivating example

We illustrate LSI in smart contract upgrades through a real-world example. On April 8, 2021, the Uranium Finance project on the Binance Smart Chain (BSC) introduced a vulnerability during an upgrade. Attackers exploited this flaw and caused a loss of about \$50 million. Figure 1 presents the source code before and after the upgrade. The vulnerability stemmed from an inconsistent modification to a core trading function in an Automated Market Maker (AMM)¹ contract. Specifically, in the original implementation (`swapV1`) shown in Fig. 1a, a scaling factor of 1000 was applied consistently to the swap output calculation and to the invariant check to account for a transaction fee. In the upgraded version (`swapV2`) shown in Fig. 1b, the multiplier in the output calculation was increased to 10000 to improve precision, yet the corresponding `require` predicate that validates the invariant remained based on 1000. This inconsistency caused the validation logic to become incompatible with the new computation rule and disabled the invariant enforcement mechanism.

This oversight enabled attackers to craft transactions that passed the outdated `require` condition while obtaining higher swap outputs under the new formula. Each execution drained liquidity from the pool because the balances were updated according to the new computation but verified against an invariant derived from the old logic. Although the arithmetic change looked minor, it changed how the same on-chain balances were interpreted and produced divergent outcomes under identical inputs and historical state. The issue therefore materialized when the upgraded logic was applied to the pre-upgrade state, and the failure became observable only through cross-version comparison under the same conditions.

Traditional vulnerability detection approaches struggle to capture inconsistencies that arise during contract upgrades. Static analyzers such as Slither and Mythril operate on single contract versions and are designed to detect common vulnerabilities like reentrancy or access control vulnerabilities. However, these tools cannot reason about behavioral equivalence between versions or evaluate whether an upgrade preserves previously valid relationships between logic and state. Even when they flag syntactic or arithmetic changes, they cannot determine whether such modifications violate assumptions embedded in historical states or invariants. Moreover, fuzzing and unit testing focus on generating new inputs for a single version, rather than reproducing historical transactions to assess consistency across upgrades. As a result, these methods fail to detect LSIs that become evident only when both versions are executed under the same inputs and state conditions.

In contrast, our framework UpgradeShield addresses this limitation by comparing pre- and post-upgrade behaviors under identical execution environments. First, it checks for RC by verifying that all original functions remain highly structurally similar (based on opcode and control-flow graph analysis using Diaphora) and that the code change is strictly additive, i.e., no original logic is removed or overwritten. RC establishes that the upgrade is intended to preserve existing behaviors. Next, UpgradeShield replays historical transactions on both contract versions under iden-

¹An AMM is a decentralized exchange mechanism that determines token prices and executes trades through deterministic mathematical formulas instead of order books.


```

1 function swapV1(uint amount0out, uint amount1out, address to,
  bytes calldata data) external lock{
2     // ...
3     uint balance0 = balance0.mul(1000).sub(0In.mul(16));
4     uint balance1 = balance0.mul(1000).sub(0In.mul(16));
5     require(balance0.mul(balance1) >= uint(reserve0).mul(reserve1
6         ).mul(1000*2), "K")
  }

```

(a) The original version of the contract (swapV1), which uses mul(1000) for calculating balance0 and balance1.

```

1 function swapV2(uint amount0out, uint amount1out, address to,
  bytes calldata data) external lock{
2     // ...
3     uint balance0 = balance0.mul(10000).sub(0In.mul(16));
4     uint balance1 = balance0.mul(10000).sub(0In.mul(16));
5     require(balance0.mul(balance1) >= uint(reserve0).mul(reserve1
6         ).mul(1000*2), "K")
  }

```

(b) The upgraded version of the contract (swapV2), which changes the multiplier to mul(10000) yet neglects to update the require statement accordingly, introducing a severe vulnerability.

Fig. 1 An example of logic-state inconsistencies triggered by a minor change in a smart contract upgrade

tical initial states and records the resulting storage states after each transaction for both versions. Finally, it performs a fine-grained comparison of critical storage slots across versions. If differences are observed in the storage state, the upgrade is flagged as exhibiting a LSI. Such LSIs reveal deep semantic divergences that elude traditional tools but can have severe consequences on contract correctness and trust.

3 Design of UpgradeShield

3.1 Challenges and solutions

Addressing LSI in smart contract upgrades involves multiple technical challenges, each of which becomes more pronounced in the context of complex contract upgrades. Below, we detail these challenges and describe how our approach addresses them:

Complexity of simulating logic and state A primary challenge in detecting LSI after a contract upgrade is the complexity of simulating logic and state. Following an upgrade, the new contract logic must operate on the previous contract's state, which may have evolved through a series of intricate and historically dependent transactions. In the case of Uranium Finance, the upgrade modified the multiplier from mul(1000) to mul(10000) in swapV2, while the require statement was not updated accordingly. This discrepancy resulted in the contract's logic being out of sync with its state, complicating the detection of the issue, particularly when the con-

tract had been involved in a variety of external interactions. Traditional fuzz testing is often inadequate in such scenarios because it struggles to simulate the complex, deep call chains and varied state conditions inherent in contract interactions, leading to insufficient test coverage and an inability to fully evaluate the contract's behavior across diverse states.

To address this challenge, we incorporate transaction replay as a central component of our framework. By replaying historical transactions on both the old and new contract versions, we ensure that the same interactions are tested under the same initial state conditions. This mechanism enables us to simulate the evolution of contract behavior over time and across various states, providing a comprehensive assessment of logic and state interactions. Through this method, we are better able to capture discrepancies in behavior that may arise due to mismatches between logic and state after the upgrade.

Complexity of understanding logic changes A second key challenge lies in the complexity of understanding logic changes during contract upgrades. These upgrades often involve adjustments to business rules, the addition of new functionalities, or optimizations of existing features. Such changes frequently depend on the characteristics of the old state. In the Uranium Finance example, the developers altered the multiplier to improve precision but overlooked updating the `require` condition. Identifying which parts of the new logic are dependent on the old state, and which are not, is a complex task, particularly when the changes between the old and new logic are substantial.

To address this challenge, we employ code clone analysis. By comparing the old and new contract versions, we can identify which parts of the code have been modified and, more critically, which changes might impact state compatibility. This focused analysis allows us to pinpoint logic changes that are sensitive to state variations, such as the change in multiplier in the Uranium Finance case, ensuring that compatibility with the old state is maintained throughout the upgrade.

Determining the authenticity of behavioral differences A third significant challenge is determining the authenticity of behavioral differences between the old and new contract versions. After an upgrade, differences in contract behavior may emerge due to changes in state dependencies or logic optimizations. Distinguishing whether these differences stem from legitimate updates or from mismatches between the function logic and storage state is crucial. In the Uranium Finance case, the mismatch between the `mul(1000)` in the `require` statement and the `mul(10000)` in the logic resulted in an exploit. However, such discrepancies are often subtle and may not be immediately apparent.

To address this challenge, we utilize differential testing alongside transaction replay. By comparing the post-transaction states between the old and new contract versions, we can detect any inconsistencies in the storage state. Coupled with the code clone analysis, we can assess whether observed behavioral differences are due

to LSI. If significant state inconsistencies are found, and the code clone between the two versions exceeds a predetermined threshold, we flag the upgrade as potentially unsafe. This methodology reduces the likelihood of false positives or negatives, ensuring that only genuine mismatches are identified.

3.2 Workflow of UpgradeShield

Figure 2 shows the overview of UpgradeShield. It mainly consists of four individual modules, namely, (1) Historical information collection (HIC); (2) Result trace generation (RTG); (3) Refactoring consistency check (RCC); (4) Result trace comparison (RTC).

In HIC, UpgradeShield re-executes all transactions that actually occurred in the history of Ethereum and records the input and output state of each transaction (i.e., the state before and after execution). The recorded historical information serves as the foundation for performing differential testing and will be used in the next phase to construct an accurate historical state. Additionally, UpgradeShield generates documents that record the bytecode of the smart contracts, along with some basic contract information, including the block number of the creation transaction and the transaction number of the creation transaction in the block, by replaying the historical

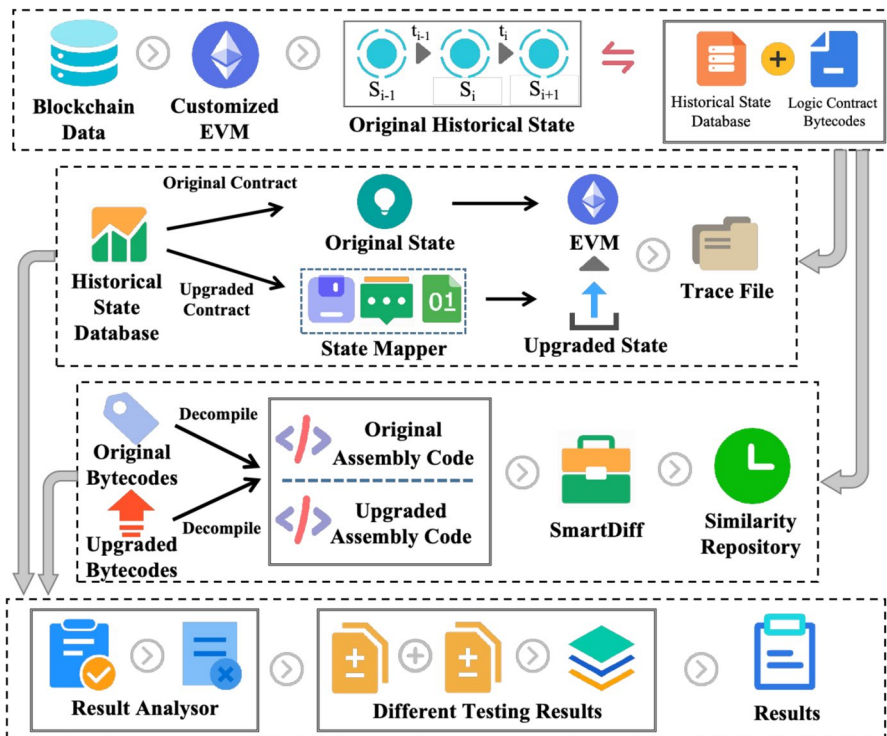


Fig. 2 Overview of UpgradeShield

information collection. The bytecodes of the contracts in each file have continuous upgrade relationships.

In RTG, UpgradeShield reads a document containing bytecodes of different contract versions and replays them in the same recorded historical state. Specifically, according to the order of contract upgrades, UpgradeShield replays contract versions step by step, with only the original version and the subsequent version replayed on the historical state at each step. The results of each replay step are recorded in a separate trace file to facilitate future comparisons.

In RCC, UpgradeShield reads the same bytecode document as the previous step and analyzes the similarity of functions between the original versions and subsequent versions to verify whether they have refactoring consistency. If the two continuous versions have refactoring consistency, they should have the same replay results. And the results of the function-level similarity analysis are stored in a database file.

In RTC, UpgradeShield reads the trace files generated in RTG and the result database file generated in RCC, and then analyzes them to determine the reliability of the contract upgrade. If two consecutive versions meet refactoring consistency but show different replay results, it indicates the presence of a LSI, except in cases where new storage keys are added without affecting the existing state variables.

4 Approach details

4.1 Historical information collection

Historical transaction information Performing differential testing on Ethereum smart contracts requires the historical world state of Ethereum, including the state of the contract, the user's account, and the transaction execution environment. However, the online execution environment can only support replay at the block granularity level, which can be time-consuming when re-executing unnecessary transactions. In order to implement differential testing more conveniently and effectively, UpgradeShield adopts a record-replay mechanism (Kim et al. 2021), which records the inputs and execution contexts of historical transactions and replays them later to reproduce identical behaviors in an offline environment. This technique enables accurate and efficient re-execution of past transactions without relying on the live blockchain state and has been proven to be both performant and precise.

To achieve this, UpgradeShield leverages a customized Geth, which denotes a modified Ethereum client to support transaction replay and fine-grained state inspection. This customization allows us to precisely track execution behavior and monitor storage-level changes across contract versions, including contract addresses, involved storage slots and their corresponding values, balances and nonces. Considering the purpose of our testing, we simplify the input state and output state of one address in a transaction as a four-tuple, namely, $\langle addr, code, \langle slot\ key, slot\ value \rangle \rangle$, where *slot key* and *slot value* refer to the key-value pairs in the EVM's storage model. To implement differential testing, UpgradeShield reconstructs two input states based on the historical input state. If the historical input state is $\langle addr, code, \langle slot\ key, slot\ value \rangle \rangle$, the input state of the original version is $\langle addr_o, code_o, \langle slot\ key_o, slot\ value_o \rangle \rangle$, and the

input state of the subsequent version is $\langle addr_o, code_m, \langle slot\ key_m, slot\ value_m \rangle \rangle$. The code and slot in the input state of the subsequent version are replaced before replay, while the addr remains the same as before.

Contract bytecode information UpgradeShield accurately restores and simulates each transaction's logic and state by collecting historical transaction information. Beyond this, it gathers two additional categories of information to refine differential testing and state analysis. The first category pertains to the creation of proxy and logic contracts, including data such as the `createbin` and `runtimebin`, block numbers, and transaction positions. The second category focuses on contract upgrades, capturing details such as the block number and transaction position of each upgrade. Bytecode-related information is used to replace the input state during differential testing and serves as input for the refactoring consistency check, ensuring the alignment of contract logic and state. Meanwhile, transaction location information defines the replay range and the scope of state collection, enabling precise state restoration.

While platforms like Etherscan typically provide contract upgrade details only at the block level, UpgradeShield operates at a finer granularity by identifying and replaying transactions individually. This granularity is crucial for accurately restoring the interactions and state changes in scenarios involving proxy contracts. For instance, as shown in Fig. 3, a single block may include multiple transactions related to a proxy contract. Transaction $Tx(i)$ represents an upgrade transaction where the logic contract invoked by the proxy contract is switched from the `old logic contract` to the `new logic contract`. Before $Tx(i)$, interactions with the proxy contract effectively use the `old logic contract`, while interactions after $Tx(i)$ invoke the `new logic contract`. If block-level granularity is used, some transactions within the block could be incorrectly assigned to the wrong logic contract, leading to inaccurate state reconstruction. By collecting transaction-specific data, UpgradeShield achieves precise control over the replay range, faithfully reproducing the behavior of historical transactions and ensuring reliable differential testing.

4.2 Result trace generation

In this module, the goal of UpgradeShield is to replay transactions across different versions of the logic contract to produce execution traces that reflect how each

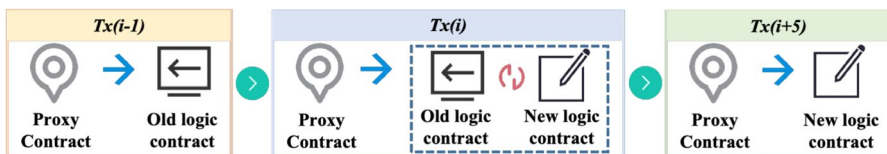


Fig. 3 Multiple proxy contract invocations within a single block: the transaction $Tx(i)$ represents a contract upgrade, altering the logic contract invoked by the proxy contract. Before the upgrade transaction, the proxy contract interacts with the old logic contract, while after the upgrade, it invokes the new logic contract

version behaves under the same set of transactions. To achieve this, UpgradeShield refines the replay scope, preserves storage states, and implements state replacement to ensure accurate trace generation.

Replay scope definition Defining an accurate *replay scope* is essential for generating precise execution traces. In UpgradeShield, the replay scope refers to the bounded set of contract versions and their associated transactions that are re-executed to compare behavioral consistency under identical initial states. Each contract upgrade forms part of an *upgrade sequence*, denoted as $\langle V_1, V_2, V_3, \dots, V_n \rangle$. This sequence represents the chronological evolution of a logic contract as it undergoes multiple upgrades through proxy mechanisms. Each V_i corresponds to one deployed logic version in the upgrade timeline. For any consecutive pair $\langle V_{i-1}, V_i \rangle$, V_{i-1} is defined as the *predecessor version*, which refers to the logic implementation before the upgrade, while V_i is defined as the *successor version*, referring to the logic implementation after the upgrade. Since storage modifications are directly affected only by adjacent versions, UpgradeShield confines differential testing to consecutive version pairs such as $\langle V_{i-1}, V_i \rangle$ and $\langle V_i, V_{i+1} \rangle$. This restriction avoids redundant comparisons across non-adjacent versions while ensuring that all upgrade-induced state changes are captured. If any significant behavioral divergence exists between V_{i-1} and V_{i+1} , it must also manifest in the intermediate storage states of either adjacent pair.

To further refine the replay scope, UpgradeShield identifies the creation transactions of the proxy and logic contracts, as well as the upgrade transactions, to determine the specific range of testing. In the proxy pattern, due to the disorder and unpredictability of logic contract upgrades, it is challenging to determine the creation order of the proxy and logic contracts. Therefore, when defining the replay scope, it is necessary to consider their creation order, and the earlier creation transaction between the proxy and logic contracts is used as the reference point.

When the upgrade sequence involves multiple versions (e.g., V_1, V_2, V_3), in addition to defining the replay scope, it is also essential to consider where the upgrade transaction occurs (assume the upgrade transaction occurs in tx_i). For all transactions within the block containing the upgrade transaction, if a transaction $tx_j < tx_i$ (i.e., the transaction number is smaller than the upgrade transaction), the state of the proxy contract should not be synchronized to the global contract state, even if tx_j involves both the proxy and logic contracts. In contrast, the state of the logic contract should always be updated, regardless of whether the transaction involves the proxy contract or whether the transaction number is smaller than the upgrade transaction.

As illustrated in Fig. 4, even if the logic contract V_i was deployed before transaction tx_i , UpgradeShield only collects transaction data related to the logic contract V_i between the deployment transaction (tx_{di}) and the upgrade transaction (tx_i) without mapping these states to the proxy contract. This approach ensures that the replay scope focuses precisely on critical points related to contract upgrades, avoiding interference from irrelevant states.

Storage states Preserving storage states is crucial during contract upgrades. Similarly, in differential testing, accurately reconstructing the corresponding historical

storage state is essential for creating effective tests. The preservation of storage states needs to focus on two key moments: after every transaction and before every upgrade.

During differential testing, UpgradeShield preserves the storage state after replaying each transaction to ensure the accuracy of detected differences. Storage offset is a typical issue in testing. If the slot value from the previous transaction is not preserved, the initial value of the offset slot remains 0, which contradicts the actual scenario. In proxy-based upgrades, contract states are stored in the proxy contract, requiring the collection of storage states from both the proxy and logic contracts. Additionally, to restore all slot values from the previous version during the upgrade, UpgradeShield records the slot values of the original version.

After completing all transactions for a specific version, UpgradeShield retains the complete storage state of the current version. This means that when replaying historical transactions of version V_{i-1} using the bytecode of V_{i-1} and V_i , the historical storage state of V_{i-1} is preserved and used as the initial state for the next step. UpgradeShield retains the complete state of V_{i-1} because using the same initial state facilitates the comparison of differences caused by bytecode changes in subsequent steps. Compared to the uncertain state of V_i , the state of V_{i-1} is deterministic and represents the actual state during the historical contract upgrade.

State replacement To test the successor version using the transactions of the predecessor version, it is necessary to replace the predecessor's storage state with that of the successor. This replacement ensures the successor version operates on its updated storage layout. The most straightforward approach is to replace all predecessor-related information with successor-related information in the transaction, including contract addresses and storage slots. However, this method is difficult to implement, particularly in proxy patterns, as the slot storing the logic contract address cannot be predicted.

To minimize the impact of replacement and achieve the stated objectives, UpgradeShield maps the predecessor's state to the successor's state before executing transactions. This mapping includes `createbin`, `runtimebin`, and slot information. In the case of proxy-based upgrades, state replacement must also consider the proxy contract's state. However, since the logic address remains unchanged, the storage slot holding the logic contract in the proxy contract becomes irrelevant, simplifying the replacement process.

We use the real-world upgrade event in the Uranium Finance project to illustrate how UpgradeShield defines and applies transaction-level replay scopes. In this case,

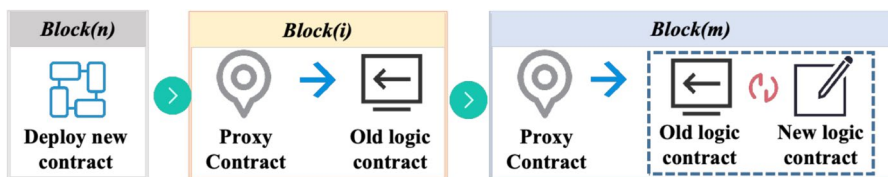


Fig. 4 Relative Positioning of Logic Contract Deployment and Upgrade Transactions

the proxy contract was redirected from `swapV1` to `swapV2` within a single block that also contained user transactions executed both before and after the upgrade. UpgradeShield precisely identifies the upgrade transaction tx_u and partitions the surrounding transactions accordingly: those preceding tx_u (e.g., tx_{u-1}) are attributed to `swapV1`, while those following it (e.g., tx_{u+1}) are attributed to `swapV2`. For each historical transaction before the upgrade, UpgradeShield performs dual re-executions under both versions of the logic contract. The replay begins from an identical initial state but proceeds independently, with each transaction applied to the state produced by its corresponding prior execution. This ensures that both versions follow their respective logic paths and state transitions, enabling precise differential testing. Furthermore, when comparing later upgrades (e.g., from `swapV2` to `swapV3`), UpgradeShield continues to anchor the initial state to the output state obtained from the `swapV1` replay. This ensures a consistent historical context across upgrade chains.

4.3 Refactoring consistency check

Before determining whether there are issues with differences in the storage states of two versions of a smart contract, it is essential to first ascertain whether these two versions should remain consistent. To this end, UpgradeShield introduces the notion of **refactoring consistency (RC)**.

RC captures a conservative assumption about developer intent: that the upgraded contract is meant to run on the legacy state without altering its externally observable behavior. This assumption is grounded in classical software engineering theory, where refactoring is defined as a transformation that improves internal code structure while preserving external behavior (Opdyke 1992; Fowler 1999). In code clone evolution, similar assumptions underlie the idea of *consistent change*, where semantically aligned edits across similar code fragments are often approximated using structural similarity (Krinke 2007; Kim et al. 2005). These concepts have recently been extended into the domain of smart contracts, where researchers have demonstrated that code clone models are useful for understanding contract maintenance and upgrade behavior (Khan et al. 2022).

The rationale for adopting the refactor consistency lies in its ability to effectively guarantee the functional stability and logical consistency of the contract. In practice, we operationalize RC by imposing two structural constraints on the upgrade:

- **Additive-only changes:** the upgraded version does not delete or overwrite any logic from the old version. This constraint ensures that all original control-flow and data-flow paths remain intact, making it possible to preserve behavior under all historical usage scenarios (Mens and Tourwé 2004).
- **High function-level similarity:** for each function in the old version, there exists a corresponding function in the new version with similarity exceeding a configurable threshold θ . This similarity is computed over opcode-token sequences. Prior empirical studies (Silva and Valente 2018; Islam et al. 2016) and smart contract-specific clone detection frameworks (Khan et al. 2022) have shown that structurally similar functions—e.g., those with identifier renaming, constant folding, or instruction reordering—typically preserve behavior with high precision.

These constraints provide a sound and explainable foundation for determining whether two versions of a contract are intended to behave identically under legacy inputs.

To verify RC at the bytecode level, UpgradeShield performs the following steps. First, it loads the contract bytecode into IDA Pro (hex rays 2018) equipped with the EVM processor module *ida-evm* (Ida processor module for the evm 2022), which disassembles the smart contract bytecode into EVM assembly instructions, each corresponding to an opcode and its operand. During this process, *ida-evm* also reconstructs function boundaries, basic blocks, and control-flow graphs, forming a structured representation suitable for analysis. Second, both contract versions are exported into IDA databases and compared using Diaphora (Diffing plugin for ida 2021), which performs function-level matching between the two versions. Specifically, Diaphora identifies pairs of functions across versions with similar opcode sequences and control-flow structures, assigning each pair a numeric similarity score. Finally, UpgradeShield checks that all matched functions satisfy both the additive-only and similarity threshold conditions, thereby establishing whether the upgrade pair satisfies RC.

We use the motivating example introduced earlier from the Uranium Finance incident to demonstrate how refactoring consistency is evaluated. In this case, the core function `swapV1` was upgraded to `swapV2` by modifying a constant multiplier (from 1000 to 10000) in the output computation logic, while preserving the surrounding control structure. No original logic was removed or overwritten, and the function maintained a high degree of structural similarity in terms of opcode sequences and control-flow graphs. Consequently, this upgrade satisfies both the additive-only constraint and the function-level similarity threshold, and is thus classified as refactoring consistent. Based on this structural evidence, the system assumes that the new version should preserve behavior for all previously valid transactions. UpgradeShield then examines the results of historical transaction replays under both versions to determine whether any LSI arise despite satisfying refactoring consistency.

4.4 Result trace comparison

In this final phase, UpgradeShield integrates the execution traces from differential testing with the similarity scores computed by Diaphora to assess whether structurally consistent upgrades exhibit behavioral divergence. The core objective is to identify inconsistencies that may arise during the execution of historical transactions under different contract versions, particularly when these versions satisfy RC. Specifically, UpgradeShield classifies detected anomalies into two categories based on their observable effects. Functional inconsistencies occur when the same transaction yields different external outputs across versions, such as return values or diverging revert behaviors. Semantic inconsistencies arise when the outputs remain identical but the corresponding storage updates differ, revealing LSIs that are invisible to output-only comparison. UpgradeShield detects functional inconsistencies by comparing the outputs of replayed transactions, while semantic inconsistencies are identified by analyzing storage-state differences when the outputs coincide.

To pinpoint these semantic differences, UpgradeShield conducts a fine-grained comparison of post-execution storage states. It classifies slot-level differences into three categories. A `dictionary_item_added` indicates that a new storage key appears in the upgraded contract but did not exist in the original version. A `dictionary_item_removed` reflects the opposite—a key present in the original version has been removed. A `values_changed` entry marks cases where the same storage key exists in both versions, but the associated values differ. These classifications allow UpgradeShield to diagnose subtle state divergences, which may affect contract correctness, fairness, or security in edge cases.

For example, in the auction contract described in our case study, the logic contract was upgraded from `0xf53f8c8d` to `0x393640ad`. Though the upgrade satisfied RC, the behavior of the `PlaceBid()` function changed under certain boundary conditions. Specifically, when a user placed a bid equal to the current highest bid, the upgraded version retained the original highest bidder, whereas the previous version would overwrite it with the new user. Although the transaction succeeded in both versions, UpgradeShield detected a `values_changed` anomaly in the storage slot holding the highest bidder address. This semantic inconsistency, invisible to output-based comparison, was surfaced only through detailed state inspection. More detailed differences and analysis can be found in Section 6.5.

In contrast, if all observed storage changes across versions are exclusively of type `dictionary_item_added`—for instance, new slots introduced for future use or extended data fields—then the upgrade is considered benign. Such additions typically indicate forward-compatible extensions that do not affect legacy behavior. Hence, when all upgrades in a proxy contract’s lifecycle satisfy RC and no LSIs are detected beyond `dictionary_item_added` entries, UpgradeShield marks the contract upgrade as trustworthy.

UpgradeShield does not require two versions to be byte-identical. An upgrade is marked as trustworthy when the version pair satisfies RC and differential replay over the same historical inputs and initial states reveals neither functional inconsistencies nor semantic inconsistencies. Changes that typically remain acceptable under this criterion include the addition of new storage keys that are not read or written by historical transactions and that do not change the interpretation of existing slots, the introduction of new functions or events that preserve the behavior of all existing functions under identical states and inputs, and internal optimizations such as gas-efficiency improvements or code refactoring that keep the control flow and write sets unchanged for historical executions. In these cases, the external outputs remain identical and the values of all preexisting storage keys remain identical across versions.

5 Implementation

Smart contracts are typically implemented using a state-based storage model, in which each contract’s persistent state is represented by a mapping from 256-bit storage keys to 32-byte values recorded on the blockchain. Each transaction deterministically transforms this mapping, producing a new state that reflects updates to the contract’s variables. Contract upgrades involve updating the code of the contract and potentially

modifying its storage variables, whether it is the storage layout or specific storage slot values. To recover the historical world state, we build UpgradeShield using a record and replay mechanism on top of an off-chain execution environment (Kim et al. 2021). Based on this mechanism, UpgradeShield can quickly construct historical world states and replay required transactions. This includes replaying transactions against the original version of a contract to collect states, as well as replaying transactions against the upgraded contract to perform differential testing for verifying the reliability of upgrades. Considering the limitations of data collection, UpgradeShield is implemented to test contract upgrades in proxy form. However, this approach can be easily adapted to other upgrade modes.

5.1 State modification

In order to replay historical transactions against the successor contract, it is necessary to replace the bytecode and storage of the predecessor version with those of the later version during execution. To minimize the impact of this replacement, UpgradeShield maps the predecessor's state to the successor's environment before executing transactions. Specifically, when executing the creation transaction of the predecessor, UpgradeShield replaces the original bytecode with the createbin of the successor to align their initialization behavior. During subsequent transaction replays, the predecessor's runtime bytecode in the reconstructed historical state is replaced with that of the successor, and the corresponding storage slots and proxy address are redirected to the successor's storage space. This mapping ensures that both versions execute under identical initial conditions, allowing UpgradeShield to isolate behavioral differences caused purely by logic changes. Because the number and usage of storage slots are unpredictable, all collected slot data are provided to the EVM during replay, which does not affect the number of slots and the value of each slot in the output.

For upgrade sequences with more than two contract versions (i.e. V_1, V_2, V_3), it is important to consider the recovery of the state. Before executing the historical transactions of V_2 using the bytecode and storage layout of V_2 and V_3 , we need to restore the state of the proxy contract. However, record-replay mechanisms only store the transaction-related historical states due to efficiency concerns. Therefore, in order to fully recover the state of the proxy contract, UpgradeShield gradually collects the states while executing the historical transactions of V_1 using the bytecode and storage layout of V_1 .

To compare behaviors under identical initial conditions, UpgradeShield constructs the historical state up to the beginning of the block that contains tx_i , and then performs dual replays. For each attributed transaction, it executes once with V_1 and once with V_2 , each run starting from the same reconstructed pre-state. Before replaying on V_2 , UpgradeShield replaces the predecessor's artifacts with those of the successor: the creation bytecode is replaced by the successor's creation bytecode for constructor phases, the runtime bytecode is replaced by the successor's runtime bytecode for normal calls, and the referenced storage is redirected to the successor's storage mapping at the same address. For transactions in the upgrade block with $tx_j < tx_i$, the replay does not commit successor-side effects to the global state used for subsequent histori-

cal transactions, ensuring that the historical ordering is preserved while still enabling cross-version comparison on identical inputs and pre-states.

For $\langle V_1, V_2, V_3 \rangle$, UpgradeShield compares consecutive pairs. It first compares $\langle V_1, V_2 \rangle$ as above, then reconstructs the historical state up to the block that contains the $V_2 \rightarrow V_3$ upgrade, and finally compares $\langle V_2, V_3 \rangle$ under the same rules.

5.2 Replay on transaction granularity

For contract upgrades in the form of a proxy, the deployment order of the proxy and its logic contracts is often uncertain, as developers may deploy the logic contract either before or after the proxy contract. To remove this ambiguity, UpgradeShield starts differential replay from the earlier of the two creation transactions (the proxy's creation or the logic contract's creation). Beginning at the earliest creation preserves the full causal history of storage initialization and pre-link writes, which is necessary to attribute later divergences to logic changes rather than to incomplete state reconstruction. To ensure that both contracts exist in the replay environment, UpgradeShield uses the later of the two creation transactions (i.e., the proxy's creation or the logic contract's creation) as the starting point of the replay scope. When it comes to upgrade sequences with more than three contract versions (i.e., V_1, V_2, V_3), in addition to considering the replay range, it is important to consider in which the implement upgrade transaction takes place (assuming the upgrade implement transaction took place in tx_i). For all transactions in the block where the upgrade transaction takes place, if the transaction $tx_j < tx_i$, the state of the proxy contract should not be synchronized to the global contract state, even if the proxy and logic contract are both included in transaction tx_j . While for the logic contract, the state of the logic contract should be updated regardless of whether the transaction contains the proxy contract and whether the transaction number is smaller than the contract upgrade transaction.

Given that contract upgrades are aimed at optimizing contracts, whether in terms of functionality, performance, or security, UpgradeShield is implemented to ensure that all historical transactions that have been successfully executed will not fail due to gas consumption.

6 Evaluation

To evaluate the effectiveness of UpgradeShield, we collected over 2000 smart contract upgrade cases that adopted the proxy pattern. For each proxy contract, we retrieved its upgrade events from Etherscan and reconstructed the sequence of logic contract versions linked to it over time. We then performed differential testing between consecutive logic contract pairs to assess behavioral consistency across upgrades. In addition, we conducted experiments to validate the parameter configuration used in the RC analysis, particularly the similarity threshold for function matching.

6.1 Evaluation setup

To evaluate the effectiveness of UpgradeShield, we analyzed 43,387 proxy contracts up to block number 18,000,000 and identified 2,015 proxy contracts that included at least one “upgrade” event. The corresponding logic contracts for these proxy contracts were retrieved using a customized Geth client and Etherscan, a widely used blockchain explorer and analytics platform for Ethereum.

To collect historical transactions and states, we extracted all encoded blockchain data up to block height 15,500,000 (1,550W)². By replaying these historical transactions, we restored and maintained the historical contract states, enabling us to analyze contract upgrade issues using differential testing effectively.

The experiments about the historical state are conducted on a machine with two Intel(R) Xeon(R) Gold 5218R CPU @ 2.10GHz, 512GB RAM, and Ubuntu 20.04.4 OS. The refactoring check and other analyses are conducted on PC. Table 1 presents other key environmental setups which are used in our evaluation.

6.2 Data collection

We analyzed a total of 43,387 proxy contracts up to block number 18,000,000, and specifically selected 2015 proxy contracts that contain at least one “upgrade” event. We first utilized the Etherscan (Ethereum: Etherscan 2023) API to obtain the upgrade events of proxy contracts. However, in some proxy contracts, the upgrade events are not fully recorded because some proxy contracts may lack their initial logic contract. To obtain a more complete upgrade sequence, we replayed the transactions to get the ‘delegatecall’ list of the proxy addresses, in which we just replayed transactions about the proxy addresses. At last, we merge the event list and ‘delegatecall’ list if there is no conflict. When there is a conflict, we use the list obtained from Etherscan. Furthermore, in order to control the process of differential testing at the transaction granularity level, we need to obtain the block number and transaction number for the creation transactions of the proxy contract and the logic contract from Etherscan. In addition, after determining the transaction sequence, we need to use customized Geth to obtain the position of the contract upgrade transaction in the creation block.

6.3 Effectiveness of UpgradeShield

We evaluate consistency under RC, which requires that each predecessor function has a sufficiently similar counterpart in the successor (similarity $\geq \theta$) and permits only additive changes. In practice, this requirement corresponds to the entries *Partial matches* and *Unmatched in primary* in Diaphora (see details in Table 2), with the similarity threshold set to $\theta = 0.79$.

As described above, we collected a total of 2,015 proxy contracts and their corresponding logic contracts. Among these, 1,444 upgrade pairs contained at least two available logic versions and were therefore eligible for differential testing.

²The term block height refers to the sequential number of a block in the Ethereum blockchain, representing its position in the chain since the genesis block.

Table 1 Environmental setup used by UpgradeShield

Server	HIC&TRG	RCC&RTC
CPU	Intel(R) Xeon(R) Gold 5218R CPU @ 2.10GHz × 2, 40 cores × 2 sockets	12th Gen Intel(R) Core(TM) i5-12400F
RAM	512 GB DDR4 RAM @3,200MHz	16 GB SO-DIMM DDR4 RAM @ 2,666MHz
SSDs PCIe	KINGSTON SFYRD4000G × 2	SAMSUNG 970 EVO Plus 500GB
OS	Ubuntu 20.04.4 LTS	Windows 10

UpgradeShield analyzed these 1,444 pairs and detected behavioral inconsistencies in 886 cases. This indicates that 61.4% of logic contract upgrades in smart contract upgrades exhibit LSIs, which may lead to potential risks after the upgrade.

To evaluate the effectiveness of UpgradeShield, we randomly sampled 178 upgrade cases from the executable subset for manual analysis. The verification process was conducted by two experienced auditors with over three years of smart contract auditing experience. A reported LSI was treated as a true positive if it met the following criteria:

- The upgrade modifies previously reachable logic, such as changes to arithmetic expressions, reordering of state updates, or adjustments to branch conditions.
- The new version exhibits an implicit behavior shift, where under the same historical state and input the transaction still succeeds but execution follows a different internal path or produces a different write set.
- No migration or state-conversion procedure is provided to reconcile the change with the pre-upgrade state.

Under these criteria, we attempted to review all 178 cases. Due to limited open-source availability, only 30 of the 178 sampled upgrade cases had fully accessible source code for both versions. Among these, 23 were confirmed as true positives based on the above criteria, resulting in an observed accuracy of 76%. Although the auditable subset is small, the sampling procedure and expert review provide a consistent basis for interpreting the results.

Thereafter, we manually reviewed the 23 confirmed true positives and classified the inconsistencies into three categories: (1) **Logic Modification**. The upgrade

Table 2 Contract-level Similarity

	Similarity	Description
1	100% equal	All functions are identical in both contracts
2	Best matches	All functions meet <i>Best matches</i> in both contracts
3	Partial matches	All functions must have corresponding counterparts in the other contract, and the similarity must exceed a certain similarity threshold
4	Partial matches and Unmatched in primary	Only function additions and functions with a similarity score above a certain similarity threshold are permissible

altered previously reachable code logic (e.g., changed arithmetic operations, reordered state updates, or modified conditional branches). (2) **Silent Behavior Shift.** The new logic introduced implicit control-flow or write-set deviations for historically valid inputs without explicit warnings. (3) **Migration Mechanism Deficiency.** The upgrade lacked proper migration procedures or state conversion steps to reconcile the changes (e.g., no backward compatibility handling). These categories reflect distinct types of semantic divergence introduced during contract upgrades and help characterize the nature of LSIs.

False positives Among the 30 inconsistencies reported by UpgradeShield in the manually audited subset, 7 were determined to be false positives under our audit criteria. Although UpgradeShield is designed to be conservative, certain upgrades can trigger differential discrepancies without violating the original semantic intent. In practice, false positives typically arose when:

- Addition of `require` statements for defensive checks that do not affect successful historical calls.
- Modifications to constant values in the source code.
- Changes confined to functions that are only callable by privileged roles (e.g., owner-only or admin-only).

False negatives Due to the lack of ground-truth datasets and the complexity of reconstructing historical contract states, identifying false negatives is inherently difficult. First, there is currently no publicly available dataset that systematically labels upgrade-induced inconsistencies at the state level, making it impossible to benchmark against known ground truth. Second, even when code changes appear minor, their effects on contract behavior often depend on intricate, long-running sequences of user interactions. Accurately detecting such inconsistencies would require full replay of transaction histories and fine-grained comparison of storage layouts, which is both labor-intensive and computationally expensive. These practical limitations make exhaustive false negative identification infeasible in real-world settings.

To partially assess the likelihood of false negatives, we manually validated 20 randomly selected upgrade pairs that were not flagged by our tool. Among these, we identified one case in which a semantic inconsistency was triggered under a rare transaction sequence. This finding suggests that while false negatives are possible, they appear to be rare in practice, and typically arise from the limitations of transaction replay fidelity rather than fundamental flaws in our detection mechanism.

Comparison with advanced tools To further validate the effectiveness of UpgradeShield, we compare it against state-of-the-art tools USCHunt (Bodell et al. 2023), Crush (Ruaro et al. 2024b), and Slither (Feist et al. 2019). The goal of this comparison is to assess whether existing tools can identify the LSIs detected by UpgradeShield under the same dataset of 886 upgrade cases. Two domain experts manually reviewed the outputs of all tools and confirmed which reports correspond to genuine LSIs based on the transaction and state evidence. Crush is a dynamic tool

that performs storage collision detection using historical transactions, but its dataset is not publicly available and its internal data structure is undocumented, preventing full reproduction. USCHunt and Slither are static analysis tools that support proxy-based checks, but they lack fine-grained execution context and storage state analysis, which makes them unable to detect LSIs.

6.4 Refactoring parameter check

To determine a reliable and practical setting for RC, UpgradeShield evaluates two configurable parameters: the function-level similarity threshold θ and whether to enforce the *only-add* constraint, which disallows deletions or overwrites during upgrades. The *Unmatched in primary* label from Diaphora's classification directly corresponds to this constraint, indicating newly added functions that do not exist in the original contract.

Table 2 summarizes four levels of contract-level similarity based on Diaphora's function matching results. Among them, the configuration *Partial matches and Unmatched in primary* directly aligns with our definition of *only-add*: no original functions are removed or rewritten, and all newly added functions must exceed the similarity threshold θ .

We manually curated 150 contract upgrade cases as our evaluation dataset. Among these, 28 were excluded due to missing contract source code, resulting in 122 upgrade pairs used for parameter analysis. Each case was independently labeled by two domain experts with over three years of smart contract auditing experience. Disagreements were resolved through iterative discussion to ensure the quality of the ground truth.

Empirical analysis reveals that satisfying *Best matches* is already an extremely strict condition—let alone achieving 100%*equal*. Among the 122 upgrade cases in our dataset, only 8 satisfy either of these two stricter conditions, indicating that such configurations would exclude the vast majority of real-world upgrades. These requirements often miss semantically equivalent modifications involving minor control-flow restructuring, auxiliary function insertion, or metadata shifts. Therefore, in the subsequent experimental analysis, we focus only on *Partial matches* and *Partial matches and Unmatched in primary*. We conduct a parameter study on this dataset by varying the similarity threshold θ from 0.01 to 1.0 (step = 0.01) under two RC configurations: (1) *Partial matches only* (strict RC, disallowing unmatched additions), and (2) *Partial matches with Unmatched in primary* (relaxed RC, allowing additions only). For each configuration and threshold, we measure: (i) the number of upgrades satisfying RC and (ii) the proportion of true positives—upgrades that do not trigger LSIs in differential testing.

To explore the overall inclusion space, we also examined the RC satisfaction rate at $\theta = 0.01$. Figure 5 presents the overall trends. When $\theta > 0.95$, both configurations yield very few matched cases, indicating poor recall. Conversely, when $\theta < 0.6$, many upgrades satisfy RC, but the proportion of true positives drops due to the inclusion of semantically inconsistent upgrades. In contrast, the relaxed RC configuration (allowing *Unmatched in primary*) consistently detects more behavior-preserv-

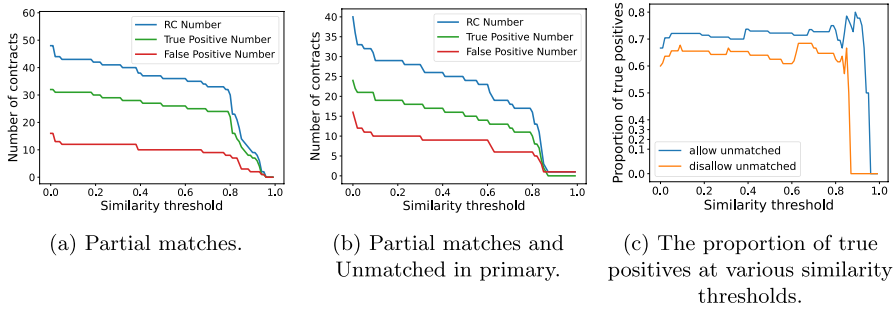


Fig. 5 The number of contracts that satisfy refactoring consistency and the number of contracts that satisfy refactoring consistency and are positive or false positive at a certain similarity threshold

ing upgrades while maintaining a comparable true positive ratio. This finding aligns with real-world practices, where upgrades often involve benign additions.

As shown in Fig. 5c, the true positive ratio peaks around $\theta = 0.79$ under the relaxed configuration. Beyond this point, the number of matching upgrades drops significantly, reducing overall coverage. Based on this trade-off between precision and recall, we adopt the configuration *Partial matches* + *Unmatched in primary* with $\theta = 0.79$ as the default setting in UpgradeShield.

6.5 Case study

We present a representative case study that illustrates how UpgradeShield surfaces a LSIs under identical historical states and inputs. The proxy at `0x8c835b3d3` upgraded its logic contract from `0xf53f8c8d4` to `0x393640ad5`. Figure 6 shows the key code edits in a simplified form. Both versions implement a function that checks and updates the current highest bid. The behavioral difference appears when a user submits a bid equal to the current highest bid (line 3 in Fig. 6(a) and (b)). In the previous version, the transaction succeeds and the address of the highest bidder is overwritten by the new user. In the upgraded version, the transaction also succeeds but the stored highest-bidder address remains unchanged.

This case highlights the fact that even minor changes in contract upgrades can have a significant impact on contract behavior, especially in complex scenarios, and can potentially mislead users. In scenarios with complex logic, some changes may trigger boundary conditions as well. Therefore, developers should explicitly state the relevant information during contract upgrades.

UpgradeShield identifies this inconsistency through differential testing after RC is verified. It first confirms that the upgrade is additive and that original functions retain high structural similarity. It then replays historical transactions on both versions under the same initial state and compares the resulting storage. In this case, the

³ `0x8c835b3df0e7d37c73a2d013331b40de41ba35f`

⁴ `0xf53f8c8d41e80eb8704eb9038f83398e70d15bc9`

⁵ `0x393640ad9767b76aadcc693c4bab42bffe8cc43c`

```

1 function PlaceBid(uint256 _auctionId, uint256 _tokenId, address
  _tokenAddress) {
2   // ...
3   if(emogramsOnAuction[_auctionId].highestBid !=
      emogramsOnAuction[_auctionId].startPrice) { {
4     // state change and return
5   } else {
6     require(emogramsOnAuction[_auctionId].highestBid < msg.
      value, "Bid too low")
7     // state change and return
8   }
9 }

```

(a) Original Contract

```

1 function PlaceBid(uint256 _auctionId, uint256 _tokenId, address
  _tokenAddress) {
2   // ...
3   if(emogramsOnAuction[_auctionId].highestBidder ==
      emogramsOnAuction[_auctionId].seller) { {
4     // state change and return
5   } else {
6     require(emogramsOnAuction[_auctionId].highestBid < msg.
      value, "Bid too low")
7     // state change and return
8   }
9 }

```

(b) Upgraded contract

Fig. 6 Code changes in contract upgrades (address:0x8c835B3Df0E7d37C73A2D013331B40DE41B a35FF)

storage slot that stores the highest bidder ($0xb0f3e905^6$) is updated in the previous version but remains unchanged in the upgraded version. By contrast, the slot that stores the bid amount ($0xb0f3e905^7$) is identical across both versions. This pattern indicates that the divergence arises from a change in winner selection logic rather than from general storage corruption. Figure 7 shows the replayed transaction and highlights the affected storage slots. The case demonstrates that UpgradeShield can detect non-trivial LSIs that occur under specific boundary conditions even when the upgrade satisfies RC and the source edits are minimal.

7 Discussion and limitations

In this section, we discuss the broader implications and limitations of UpgradeShield based on the large-scale evaluation results. We also analyze irregular upgrade behaviors observed in the dataset, which provide additional insight into practical upgrade governance and future research directions.

⁶0xb0f3e90519fd0de3100ca1a24094c0a25e5cbf467d74a2603172b5a5204555c9

⁷0xb0f3e90519fd0de3100ca1a24094c0a25e5cbf467d74a2603172b5a5204555cb

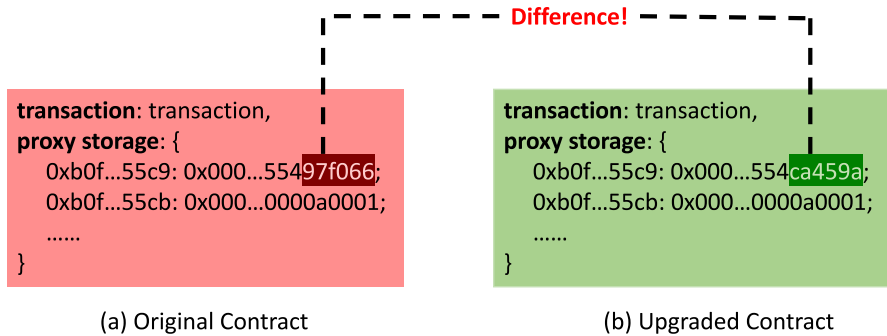


Fig. 7 State Difference

7.1 Irregular upgrade behaviors observed in practice

Beyond detecting LSIs, applying UpgradeShield at scale also reveals irregular upgrade behaviors in the wild. These behaviors are not severe security issues, yet they introduce operational inefficiencies, waste gas, and may confuse auditors and users about the actual upgrade state.

First, we identify redundant consecutive upgrades at $0x637c871c^8$, where the proxy was upgraded twice to the same implementation address. Figure 8 lists the relevant events and transactions. There was no invocation between the two upgrades, so the second operation did not change contract behavior and consumed gas without benefit.

Second, we observe upgrades that change the implementation address but are never invoked through `DELEGATECALL`. An example appears at $0xd03a860f^9$. In this scenario, the recorded upgrade modifies the implementation pointer, yet subsequent activity does not exercise the new code path.

These patterns do not constitute vulnerabilities in our samples. They nonetheless indicate avoidable costs and maintenance ambiguity. In practice, simple hygiene checks that flag redundant self-upgrades and never-invoked implementations complement LSI-oriented analyses and help operators keep upgrade workflows economical, predictable, and auditable.

7.2 Interpreting behavioral divergences under RC

In this work, UpgradeShield adopts a conservative stance: under the assumption of RC, any semantic deviation in storage state or output behavior is treated as a LSI. This design choice is motivated by two practical constraints.

First, in most real-world settings, smart contract upgrades are poorly documented. While some projects offer informal descriptions of changes, there is rarely any formal or versioned specification that indicates which behavioral divergences are intentional and which are accidental (Li et al. 2020; Zhao et al. 2022). Without reliable upgrade

⁸ $0x637c871c559ade45b37074fcf3b8081ec81c55fc$

⁹ $0xd03a860f481e83a8659640dc75008e9fcd5d879$

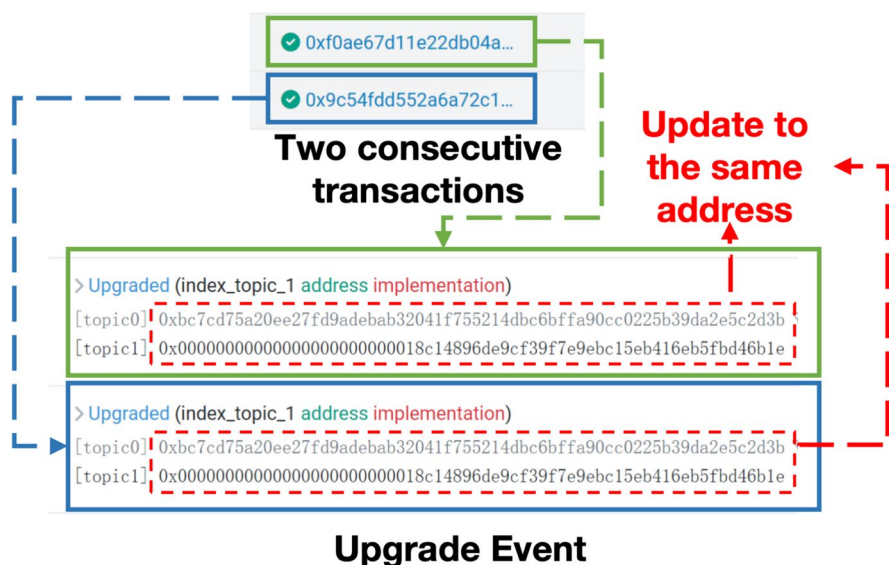


Fig. 8 An unreasonable contract upgrade behavior involving consecutive upgrades to the same address, leading to redundancy, gas wastage, and potential misinterpretation

notes or changelogs, it becomes difficult—even for human auditors—to distinguish refactorings from functional shifts, particularly when subtle control-flow or storage changes are involved.

Second, although recent research has explored the automated inference of contract “intent” from bytecode or source-level patterns, such methods still yield low precision and recall in practice. For example, Pan et al. report that even state-of-the-art symbolic alignment approaches often fail to extract meaningful behavior summaries from obfuscated or compiler-optimized bytecode (Pan et al. 2023). Similarly, Chloe (Kalra et al. 2022) shows that learning specifications from source-level traces remains brittle in the absence of rich annotations.

Given these constraints, we conservatively treat RC-breaking behavior as potentially erroneous. We acknowledge that this strategy may report some meaningful upgrades as LSIs. However, from the user’s perspective, any silent behavioral change—regardless of intent—can result in unexpected consequences if not properly announced, migrated, or verified. By flagging such cases, UpgradeShield provides a safety net that encourages developers to document and test upgrades more rigorously.

7.3 Threats to validity

Internal validity The core mechanism of UpgradeShield is to detect LSI based on the replay of historical transactions. This approach brings several inherent limitations. In detail, if a function has never been called or has been called very few times in historical transactions, even if there are semantic errors in the function or if an upgrade introduces behavioral changes, UpgradeShield may not capture these changes. This can

lead to the tool missing inconsistencies at the function level. Besides, UpgradeShield uses code clone detection to compare functions before and after an upgrade. If the upgrade involves significant changes, noticeable structural modifications, or includes obfuscation techniques (such as variable renaming or extensive refactoring), the similarity between functions may fall below the set threshold, resulting in clone identification failure and the inability to conduct differential testing. Finally, if a developer directly removes a specific existing function during an upgrade, UpgradeShield may not find a corresponding function in the new version for comparison. Such deletion actions, if logic or state handling mechanisms are not migrated synchronously, may still introduce latent defects that current tools cannot effectively identify.

External validity The dataset we select only includes contracts with multiple versions and open-source code, which are typically developed actively and have mature projects. As a result, this may underestimate the actual occurrence rate of upgrade errors in projects with closed-source code, poor maintenance, or early-stage projects.

8 Related work

Vulnerability detection of smart contract Symbolic execution pseudo-executes programs by replacing unspecified information in the program itself with abstract symbolic values to represent any value (Luu et al. 2016; Torres et al. 2018; Chen et al. 2021; Mossberg et al. 2019; Nikolić et al. 2018). For example, Oyente takes the bytecode of a smart contract as input and can detect vulnerabilities. DefectChecker uses heap events and feature detectors instead of an SMT solver, thus making itself superior to other symbolic execution-based methods in terms of speed and accuracy. Static analysis, which involves scanning the program code through thorough analysis without executing the program (Tsankov et al. 2018; Tikhomirov et al. 2018; Bose et al. 2022; Rodler et al. 2019). Sailfish uses a hybrid approach to automatically find state-inconsistency bugs in smart contracts. Dynamic analysis detects whether there are security vulnerabilities by observing the behavior of the program running under different data (Jiang et al. 2018; Zhang et al. 2020; Nguyen et al. 2020; Wüstholtz and Christakis 2020; Su et al. 2022; Liu et al. 2023; Ye et al. 2024; Zhang et al. 2025). ContractFuzzer generates input data using the ABI of the smart contract. Finally, the running track recorded by EVM is used for vulnerability analysis. Sfuzz is based on the traditional tool AFL, and adopts a feedback adaptive fuzzy strategy to improve path coverage. Midas explores profitable vulnerabilities through transaction generation. These techniques largely analyze a single contract version. They do not check whether behavior remains consistent across upgrades under the same historical state and inputs, which is the focus of our work.

Contract upgrades, as an important means of ensuring contract reliability, are receiving increasing attention regarding their own security (Bodell et al. 2023; Salehi et al. 2022; Liu et al. 2024; Bui et al. 2021; Li et al. 2024; Ruaro et al. 2024b; Zhang et al. 2025; Feist et al. 2019). USCDetector identifies upgradeable smart contracts and constructs upgrade chains without source code, and uses them to conduct a large-

scale measurement study on over 60 million contracts. Slither is a static framework designed to provide granular information about smart contract code and the flexibility necessary to support contract upgrade checks. CRUSH is a system that detects and exploits storage collision vulnerabilities across smart contracts. SmartUpdater, a hyperproxy-based toolchain for automated, secure, and transparent smart contract maintenance that handles both logic and state contract updates with reduced gas costs. Existing upgrade analyses emphasize identification, storage layout safety, or maintenance tooling. They do not integrate historical transaction replay with cross-version state comparison to detect logic–state inconsistencies that arise only when new logic interacts with preserved state.

The UpgradeShield framework uses historical transactions to replay different versions of contracts, which can be considered a type of fuzz testing. However, unlike other fuzz testing approaches, UpgradeShield focuses on issues caused by changes between different versions of the contract, rather than vulnerabilities that can be detected from a single contract. While previous research in the field has touched upon issues related to contract version changes, the issues mentioned are still detectable from a single version. Slither can detect the issues between two versions from the perspective of storage conflicts. However, some complex and subtle issues cannot be resolved by Slither.

Replaying in blockchain Replaying historical data can serve diverse purposes in blockchain technology, including analyzing historical data through Ethereum replay. Hartel and Van Staalduinen (2019) proposed a historical transaction generation approach based on Truffle, through which they reproduced the historical data of smart contracts on the blockchain and demonstrated the transparency of the blockchain. Kim et al. (2021) developed a fast off-chain replay tool which is based on a record-replay mechanism. And Wu et al. (2022) also implemented a tool that supported fast replay of historical transaction data for analyzing attack cases. However, due to the lack of accuracy or corresponding mechanisms, these works can only support the replay of historical data. Replaying transactions can also be used to discover vulnerabilities in smart contracts and verify the reliability of methods. ContractFuzzer (Jiang et al. 2018) and EVMFuzzer (Fu et al. 2019) use fuzzing differential testing to discover the vulnerabilities of Ethereum smart contracts and EVM, respectively. Moreover, replaying transactions can be used to analyze the dynamic behavior of the Ethereum ecosystem. The work of Yang et al. (2019) used replaying historical data to analyze the relationship between transaction resource consumption and gas cost and found that the resource consumption to gas ratio of instructions is inconsistent, which may cause some DOS attacks and threaten the Ethereum ecosystem. TokenScope (Chen et al. 2019) also investigates inconsistencies in tokens in Ethereum through replaying transactions. Prior replay-based systems reconstruct execution for analysis or testing, yet they do not couple replay with refactoring consistency checks and fine-grained storage comparison across versions. As a result, they cannot directly reveal upgrade-induced logic–state inconsistencies under identical historical conditions.

9 Conclusion

In this paper, we propose UpgradeShield, the first differential tool to detect bugs and issues introduced by contract upgrades. UpgradeShield is based on a record-replay mechanism and faithfully records historical transactions and bytecode of logic contracts in the record phase. And UpgradeShield replays the different bytecodes of the logic contracts on the same historical transaction to execute the differential test. By comparing trace results that adhere to refactoring consistency, UpgradeShield can verify the effectiveness of upgrades and detect problems and bugs introduced by upgrades, thus increasing the credibility of contract upgrades. The setting of refactoring consistency is based on relevant literature on code clones, and the parameters used in UpgradeShield are chosen by comparing different refactoring consistency settings. Through our experiments, we demonstrate the reliability of UpgradeShield in the context of contract upgrades, which can benefit the smart contract ecosystem by providing a trustworthy tool for evaluating smart contract upgrades and ensuring the safety and reliability of smart contract upgrades.

Acknowledgments This work was supported in part by the National Key Research and Development Program of China (2023YFB2704100), the National Natural Science Foundation of China (No. 62572497, 62372485, 62332004), the NSFC-RGC Collaborative Research (No. 62461160332), Guangdong Zhujiang Talent Program (No. 2023QN10X561).

Author contributions Wei, Yuxin, Yuhong and Zibin designed the methodology and developed the framework. Wei, Kaiwen and Jingwen were responsible for data collection and processing. Wei performed code modifications and contributed to the technical implementation. Kaiwen, Jingwen and Yuhong prepared all figures and visualizations. Wei wrote the main manuscript text, with input and critical revisions from all authors. All authors reviewed and approved the final version of the manuscript.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

- Apostolou, D.: Upgrading smart contracts (2023). <https://ethereum.org/en/developers/docs/smart-contracts/upgrading/>
- Bodell III, W.E., Meisami, S., Duan, Y.: Proxy hunting: understanding and characterizing proxy-based upgradeable smart contracts in blockchains. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 1829–1846 (2023)
- Bose, P., Das, D., Chen, Y., Feng, Y., Kruegel, C., Vigna, G.: Sailfish: Vetting smart contract state-inconsistency bugs in seconds. In: 2022 IEEE Symposium on Security and Privacy (SP). pp. 161–178. IEEE (2022)
- Bui, V.C., Wen, S., Yu, J., Xia, X., Haghighi, M.S., Xiang, Y.: Evaluating upgradable smart contract. In: 2021 IEEE International Conference on Blockchain (Blockchain), pp. 252–256. IEEE (2021)
- Chen, J., Xia, X., Lo, D., Grundy, J., Luo, X., Chen, T.: Defectchecker: Automated smart contract defect detection by analyzing evm bytecode. IEEE Trans. Softw. Eng. (2021)

- Chen, T., Zhang, Y., Li, Z., Luo, X., Wang, T., Cao, R., Xiao, X., Zhang, X.: Tokenscope: Automatically detecting inconsistent behaviors of cryptocurrency tokens in ethereum. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, pp. 1503–1520 (2019)
- Cointelegraph: Hacker drains \$1.08m from audius following passing of malicious proposal (2022). <https://cointelegraph.com/news/hacker-drains-1-08m-from-audius-following-passing-of-malicious-proposal>
- Diffing plugin for ida (2021). <https://github.com/joxeankoret/diaphora/tree/diaphora-1.2>
- Ethereum: Etherscan (Accessed 2023). <https://etherscan.io/>
- Feist, J., Grieco, G., Groce, A.: Slither: a static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), pp. 8–15. IEEE (2019)
- Finance, U.: The article about the exploit of uranium finance (2021). <https://uraniumfinance.medium.com/exploit-d3a88921531c>
- Fowler, M.: Refactoring: Improving the Design of Existing Code. Addison-Wesley (1999)
- Fu, Y., Ren, M., Ma, F., Shi, H., Yang, X., Jiang, Y., Li, H., Shi, X.: Evmfuzzer: detect evm vulnerabilities via fuzz testing. In: Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering, pp. 1110–1114 (2019)
- Gabriel Barros, P.G.: Erc-1822: Universal upgradeable proxy standard (uups) (2019). <https://eips.ethereum.org/EIPS/eip-1822>
- Halborn: Explained: The nomad hack (august 2022) (2022) <https://www.halborn.com/blog/post/explained-the-nomad-hack-august-2022>
- Hartel, P., van Staaldin, M.: Truffle tests for free-replaying ethereum smart contracts for transparency. arXiv preprint [arXiv:1907.09208](https://arxiv.org/abs/1907.09208) (2019)
- hex rays: The ida pro disassembler and debugger (2018). <https://hex-rays.com/ida-pro/>
- Ida processor module for the evm (2022). <https://github.com/crytic/ida-evm>
- Islam, M.M.H., Sajani, H., Lopes, C.V.: Sourcererc: Scaling code clone detection to big code. In: Proceedings of the 38th International Conference on Software Engineering (ICSE). pp. 1157–1168 (2016)
- Jiang, B., Liu, Y., Chan, W.K.: Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In: 2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE). pp. 259–269. IEEE (2018)
- Jorge Izquierdo, M.A.: Erc-897: Delegateproxy (2018). <https://eips.ethereum.org/EIPS/eip-897>
- Kalra, S., Goel, S., Arya, V.: Chloe: Specification inference for smart contracts. In: Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). pp. 395–405 (2022)
- Kalra, S., Goel, S., Dhawan, M., Sharma, S.: Zeus: analyzing safety of smart contracts. In: Ndss. pp. 1–12 (2018)
- Khan, F., David, I., Varro, D., McIntosh, S.: Code cloning in smart contracts on the ethereum platform: An extended replication study. IEEE Trans. Softw. Eng. (2022)
- Kim, Y., Jeong, S., Jezek, K., Burgstaller, B., Scholz, B.: An off-the-chain execution environment for scalable testing and profiling of smart contracts. In: USENIX Annual Technical Conference, pp. 565–579 (2021)
- Kim, M., Sazawal, V., Notkin, D., Murphy, G.: An empirical study of code clone genealogies. In: Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 187–196 (2005)
- Krinke, J.: A study of consistent and inconsistent changes to code clones. In: Proceedings of the 14th Working Conference on Reverse Engineering (WCRE), pp. 170–178 (2007)
- Li, S., Su, Z.: Finding unstable code via compiler-driven differential testing. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 3, pp. 238–251 (2023)
- Li, X., Yang, J., Chen, J., Tang, Y., Gao, X.: Characterizing ethereum upgradable smart contracts and their security implications. In: Proceedings of the ACM Web Conference 2024, pp. 1847–1858 (2024)
- Li, W., Shi, Y., Wang, X., et al.: A survey on smart contract development and security. IEEE Access 8, 131486–131506 (2020)
- Liu, Y., Li, S., Wu, X., Li, Y., Chen, Z., Lo, D.: Demystifying the characteristics for smart contract upgrades. arXiv preprint [arXiv:2406.05712](https://arxiv.org/abs/2406.05712) (2024)
- Liu, Z., Qian, P., Yang, J., Liu, L., Xu, X., He, Q., Zhang, X.: Rethinking smart contract fuzzing: Fuzzing with invocation ordering and important branch revisiting. arXiv preprint [arXiv:2301.03943](https://arxiv.org/abs/2301.03943) (2023)

- Luu, L., Chu, D.H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp. 254–269 (2016)
- Mens, T., Tourwé, T.: A survey of software refactoring. *IEEE Trans. Software Eng.* **30**(2), 126–139 (2004)
- Mossberg, M., Manzano, F., Hennenfent, E., Groce, A., Grieco, G., Feist, J., Brunson, T., Dinaburg, A.: Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1186–1189. IEEE (2019)
- Mudge, N.: Erc-1538: Transparent contract standard (2018). <https://eips.ethereum.org/EIPS/eip-1538>
- Mudge, N.: Erc-2535: Diamonds, multi-facet proxy (2020). <https://eips.ethereum.org/EIPS/eip-2535>
- Mueller, B.: Smashing ethereum smart contracts for fun and real profit. HITB SECONF Amsterdam **9**(54), 4–17 (2018)
- Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T.: sfuzz: An efficient adaptive fuzzer for solidity smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 778–788 (2020)
- Nikolić, I., Kolluri, A., Sergey, I., Saxena, P., Hobor, A.: Finding the greedy, prodigal, and suicidal contracts at scale. In: Proceedings of the 34th Annual Computer Security Applications Conference, pp. 653–663 (2018)
- Opdyke, W.F.: Refactoring Object-Oriented Frameworks. Ph.D. thesis, University of Illinois at Urbana-Champaign (1992)
- Openzeppelin: Proxy patterns (2023). <https://docs.openzeppelin.com/updates-plugins#proxy-pattern>
- OpenZeppelin: Upgrading smart contracts (2023). <https://docs.openzeppelin.com/learn/upgrading-smart-contracts?pref=hardhat#whats-in-an-upgrade>
- Palladino, S., Giordano, F., Croubois, H.: Erc-1967: Proxy storage slots (2019). <https://eips.ethereum.org/EIPS/eip-1967>
- Pan, X., et al.: Formal intent inference from smart contract bytecode. In: Proceedings of the IEEE Symposium on Security and Privacy (S&P) (2023)
- Rodler, M., Li, W., Karame, G.O., Davi, L.: Sereum: Protecting existing smart contracts against re-entrancy attacks. In: Proceedings of 26th Annual Network & Distributed System Security Symposium (NDSS), (2019). <http://tubiblio.ulb.tu-darmstadt.de/111410/>, event Title: 26th Annual Network & Distributed System Security Symposium (NDSS)
- Ruaro, N., Gritti, F., McLaughlin, R., Grishchenko, I., Kruegel, C., Vigna, G.: Not your type! detecting storage collision vulnerabilities in ethereum smart contracts. In: Network and Distributed System Security Symposium (2024)
- Ruaro, N., Gritti, F., McLaughlin, R., Grishchenko, I., Kruegel, C., Vigna, G.: Not your type! detecting storage collision vulnerabilities in ethereum smart contracts. In: Proceedings of the Network and Distributed System Security Symposium, pp. 1–17 (2024)
- Salehi, M., Clark, J., Mannan, M.: Not so immutable: Upgradeability of smart contracts on ethereum. In: International Conference on Financial Cryptography and Data Security, pp. 539–554. Springer (2022)
- Silva, D., Valente, M.T.: Refdiff: Detecting refactorings in version histories. *IEEE Trans. Software Eng.* **44**(12), 1216–1240 (2018)
- Su, J., Dai, H.N., Zhao, L., Zheng, Z., Luo, X.: Effectively generating vulnerable transaction sequences in smart contracts with reinforcement learning-guided fuzzing. In: 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–12 (2022)
- Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., Alexandrov, Y.: Smart-check: Static analysis of ethereum smart contracts. In: Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain, pp. 9–16 (2018)
- Torres, C.F., Iannillo, A.K., Gervais, A., State, R.: Confuzzius: A data dependency-aware hybrid fuzzer for smart contracts. In: 2021 IEEE European Symposium on Security and Privacy (EuroS&P), pp. 103–119. IEEE (2021)
- Torres, C.F., Schütte, J., State, R.: Osiris: Hunting for integer bugs in ethereum smart contracts. In: Proceedings of the 34th Annual Computer Security Applications Conference, pp. 664–676 (2018)
- Tsankov, P., Dan, A., Drachler-Cohen, D., Gervais, A., Buenzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 67–82 (2018)
- Vitalik Buterin, C.R.: Eip-214: New opcode staticcall (2017). <https://eips.ethereum.org/EIPS/eip-214>

- Wang, Y., Lahiri, S.K., Chen, S., Pan, R., Dillig, I., Born, C., Naseer, I., Ferles, K.: Formal verification of workflow policies for smart contracts in azure blockchain. In: *Verified Software. Theories, Tools, and Experiments: 11th International Conference, VSTTE 2019, New York City, NY, USA, July 13–14, 2019, Revised Selected Papers 11*, pp. 87–106. Springer (2020)
- Wood, G.: Ethereum: A secure decentralised generalised transaction ledger berlin version beacfbdb – 2022–10-24 (2020). <https://ethereum.github.io/yellowpaper/paper.pdf>
- Wood, G.: Ethereum: A Secure Decentralised Generalised Transaction Ledger. Ethereum Project (2014)
- Wüstholtz, V., Christakis, M.: Harvey: A greybox fuzzer for smart contracts. In: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1398–1409 (2020)
- Wu, S., Wu, L., Zhou, Y., Li, R., Wang, Z., Luo, X., Wang, C., Ren, K.: Time-travel investigation: toward building a scalable attack detection framework on ethereum. *ACM Trans. Softw. Eng. Methodol. (TOSEM)* **31**(3), 1–33 (2022)
- Xue, Y., Ye, J., Zhang, W., Sun, J., Ma, L., Wang, H., Zhao, J.: xfuzz: Machine learning guided cross-contract fuzzing. *IEEE Trans. Depend. Secure Comput.* (2022)
- Yang, X., Chen, Y., Eide, E., Regehr, J.: Finding and understanding bugs in c compilers. In: *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 283–294 (2011)
- Yang, R., Murray, T., Rimba, P., Parampalli, U.: Empirically analyzing ethereum’s gas mechanism. In: *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 310–319. IEEE (2019)
- Ye, M., Lin, X., Nan, Y., Wu, J., Zheng, Z.: Midas: Mining profitable exploits in on-chain smart contracts via feedback-driven fuzzing and differential analysis. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 794–805 (2024)
- Zhang, X., Song, Y., Du, Y., Cai, C., Cheng, H., Xu, K., Li, Q.: Smartupdater: Enabling transparent, automated, and secure maintenance of stateful smart contracts. *IEEE Trans. Softw. Eng.* (2025)
- Zhang, M., Zhang, X., Zhang, Y., Lin, Z.: Txspectator: Uncovering attacks in ethereum from transactions. In: *29th USENIX Security Symposium (USENIX Security 20)*, pp. 2775–2792 (2020)
- Zhang, J., Zheng, Z., Nan, Y., Ye, M., Ning, K., Zhang, Y., Zhang, W.: SmartReco: Detecting Read-Only Reentrancy via Fine-Grained Cross-DApp Analysis. In: *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 594–594 (2025)
- Zhao, B., Liu, Y., Su, Z.: SpecsSync: Automated change impact analysis for smart contracts. In: *Proceedings of the ACM Conference on Computer and Communications Security (CCS)* (2022)

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Wei Li¹ · Yuxin Su¹ · Yuhong Nan¹ · Kaiwen Ning^{1,2} · Jingwen Zhang^{1,2} · Jiajing Wu¹ · Zibin Zheng¹

✉ Wei Li
liwei378@mail2.sysu.edu.cn

✉ Yuxin Su
suyx35@mail.sysu.edu.cn

✉ Yuhong Nan
nanyh@mail.sysu.edu.cn

✉ Kaiwen Ning
ningkw@mail2.sysu.edu.cn

✉ Jingwen Zhang
zhangjw273@mail2.sysu.edu.cn

✉ Jiajing Wu
wujiajing@mail.sysu.edu.cn

✉ Zibin Zheng
zhzibin@mail.sysu.edu.cn

¹ School of Software Engineering, Sun Yat-Sen University, No.2 Daxue Road, ZhuHai 510275, China

² Peng Cheng Laboratory, No.6001 Shahexi Road, ShenZhen, ShenZhen 518057, China